

Evolution of Garbage Collection in Java Virtual Machine

LOGEEK NIGHT – February 2015

Ionuț Baloșin



Evolution of Garbage Collection in Java Virtual Machine

About

◆ Ionuț Baloșin

technical leader at [Luxoft.com](https://luxoft.com)

8 years of Java experience

- ◆ 3 years of Java financial applications

author of "*Java Performance And Tuning*" – [Luxoft-Training.com](https://luxoft-training.com) course

Evolution of Garbage Collection in Java Virtual Machine

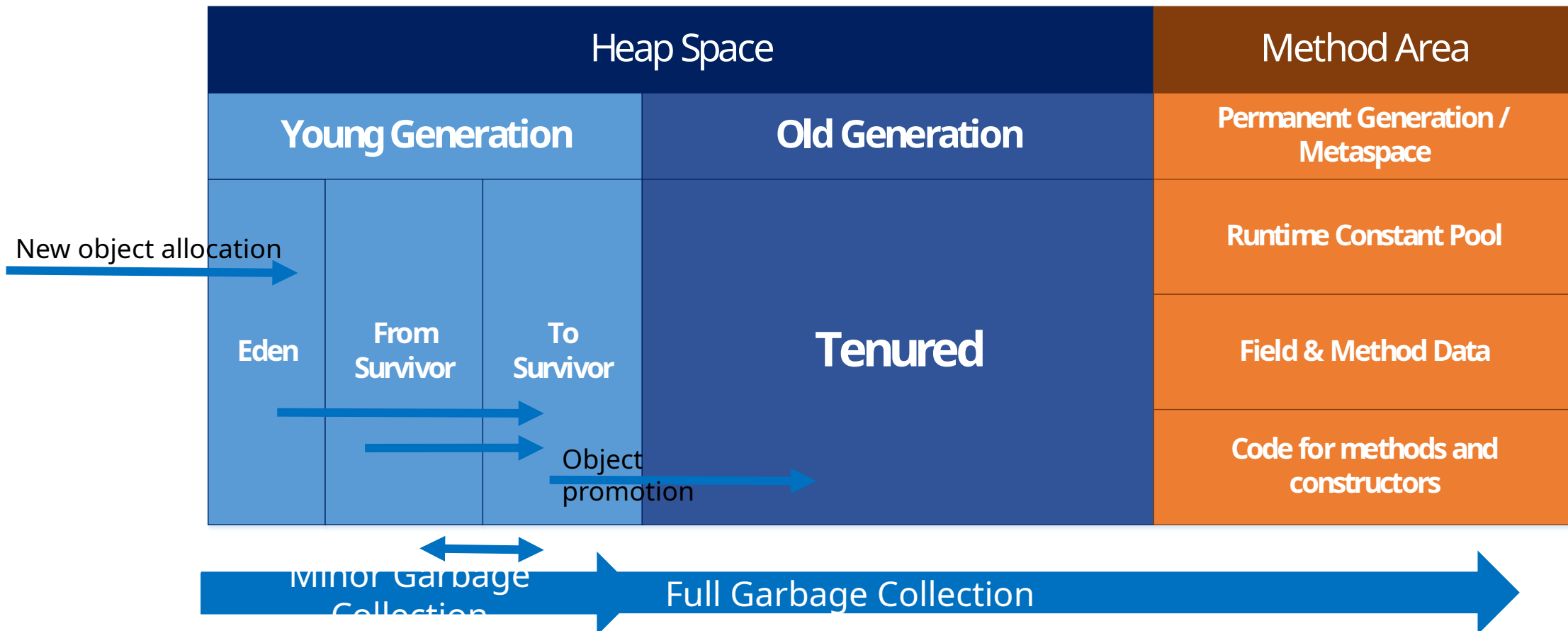
Agenda

- Java HotSpot VM Memory Layout
- Garbage Collectors
- Advices of choosing the appropriate Garbage Collector
- Garbage Collection Viewer tool
- Garbage Collection friendly programming

❑ Quizzes

Evolution of Garbage Collection in Java Virtual Machine

Java Hot Spot VM Memory Layout



Evolution of Garbage Collection in Java Virtual Machine

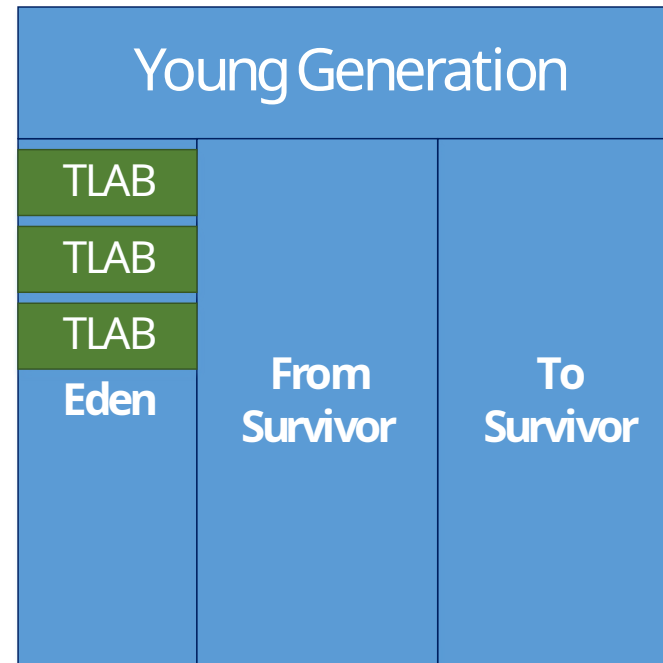
Garbage Collectors

- ♦ Frequency of minor GC is dictated
 - Application object allocation rate
 - Size of the **Eden** space
- ♦ Frequency of object promotion into old generation is dictated
 - Frequency of minor GCs (how quickly objects age)
 - Size of the **Survivor** spaces
- ♦ Ideal situation
 - promote as little as possible
 - never experience need for **Old** generation collection
 - ♦ Minor GCs are (generally) the fastest GC

Evolution of Garbage Collection in Java Virtual Machine

Quizz

- ♦ How would you deal with concurrent Eden allocations from different threads?



- ♦ **Thread Local Allocation Buffers (TLAB)**

used by default in JVM

is pre-allocated for each thread

when a thread needs to allocate memory in TLAB, it uses “**bump the pointer**” allocation within that area

Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

- ◆ Choices for designing/selecting a GC algorithm:

Serial vs. **Parallel**

Concurrent vs. **Stop-the-World**

Compacting vs. **Non-Compacting** vs. **Copying**

Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

1. Serial GC
2. Parallel GC
3. Parallel Old GC (Parallel Compacting GC)
4. Concurrent Mark & Sweep GC (or "CMS")
5. Garbage First (G1) GC

Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

1. Serial GC

Young Generation

- ♦ Stop-the-world serial Scavenger

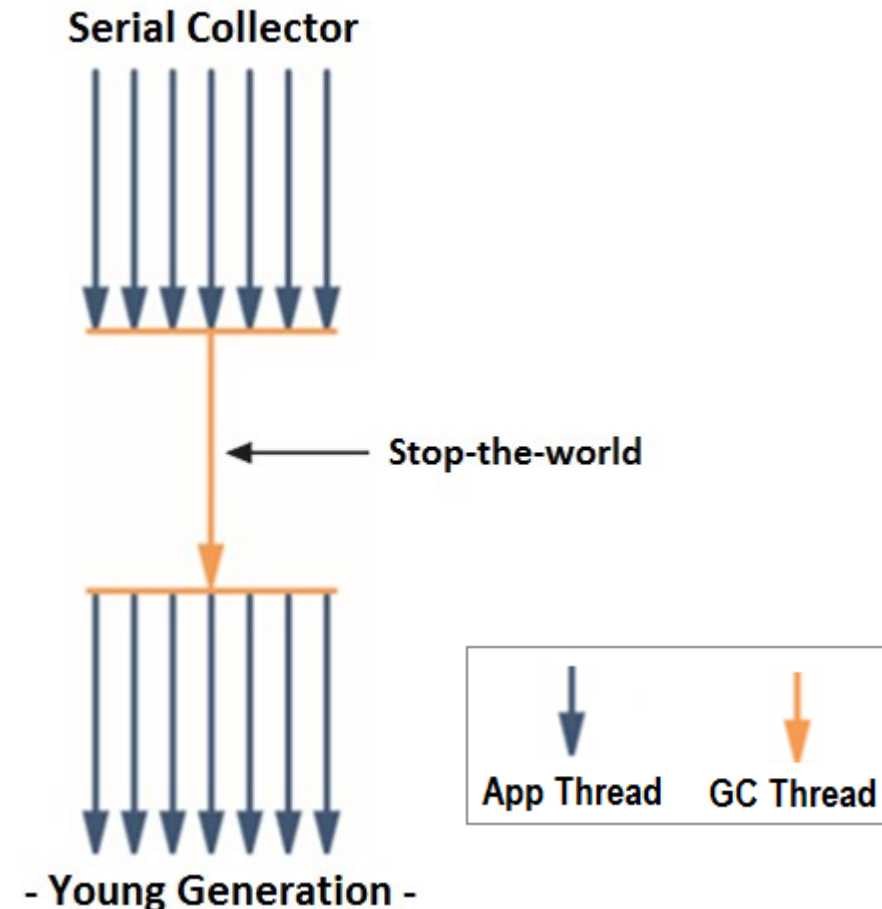
Old Generation

- ♦ Stop-the-world serial [Mark-Sweep-Compact](#)

When to Use

- ♦ for single CPU core and relatively smaller heaps (~1 GB)

Serial Collector selection: [-XX:+UseSerialGC](#)



Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

Parallel GC

Young Generation

- ◆ Stop-the-world parallel Scavenger

Old Generation

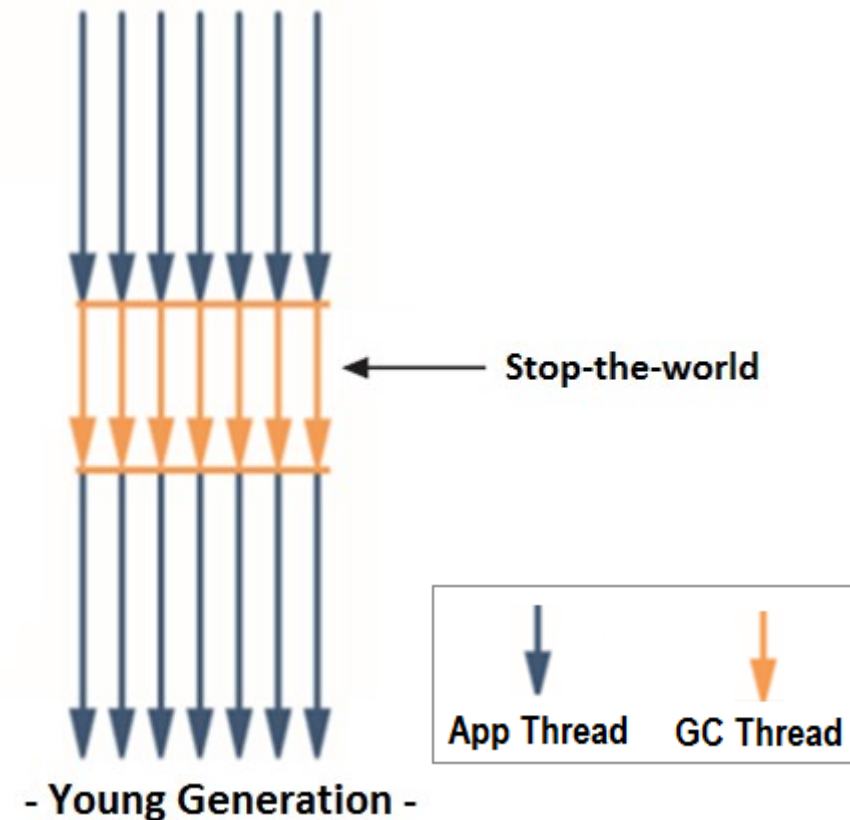
- ◆ Stop-the-world serial [Mark-Sweep-Compact](#)

When to Use

- ◆ for applications with a requirement for more than one CPU core but still small heaps (~ 1-2 GB)
- ◆ more suitable for applications do not have pause time constraints

Parallel Collector selection: `-XX:+ UseParallelGC`

Parallel Collector



Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

3. Parallel Old GC (Parallel Compacting GC)

Young Generation

- ♦ Stop-the-world parallel Scavenger

Old Generation done in parallel threads

- ♦ Stop-the-world parallel [Mark-Summary-Compact](#)

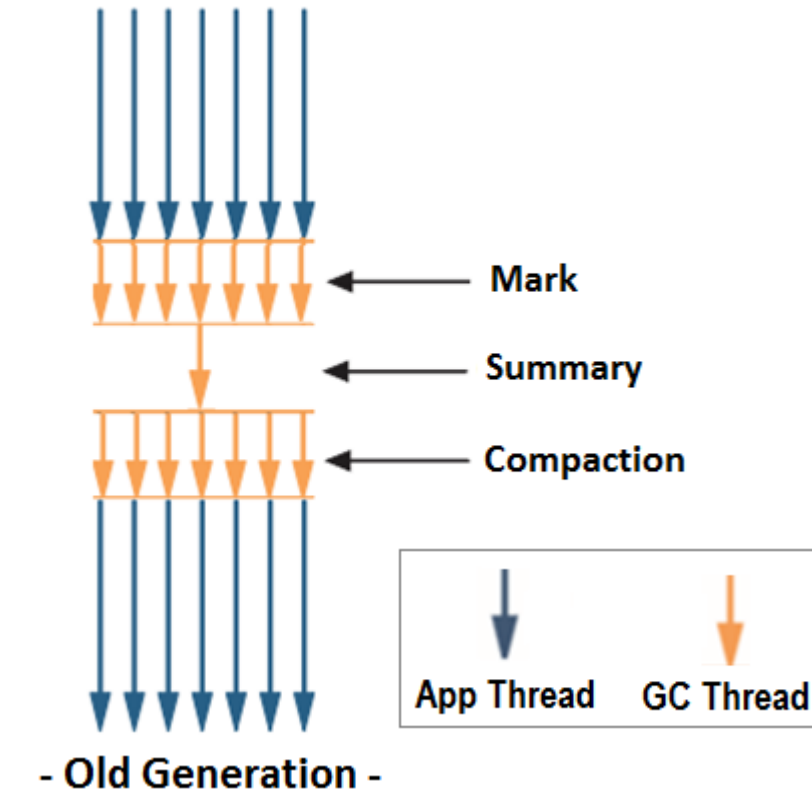
Advantage:

- ♦ faster object promotion due to Promotional Local Allocation Buffers (PLAB)

When to Use

- ♦ good throughput for 2 or more CPU cores with medium (up to 4 GB) heaps
- ♦ more suitable than [Parallel Collector](#) for applications that have pause time constraints

Parallel Old Collector



Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

4. Concurrent Mark & Sweep GC (or "CMS")

Young Generation

- ◆ Stop-the-world parallel scavenger

Old Generation

- ◆ Concurrent and parallel [Mark-Sweep](#)

Disadvantage:

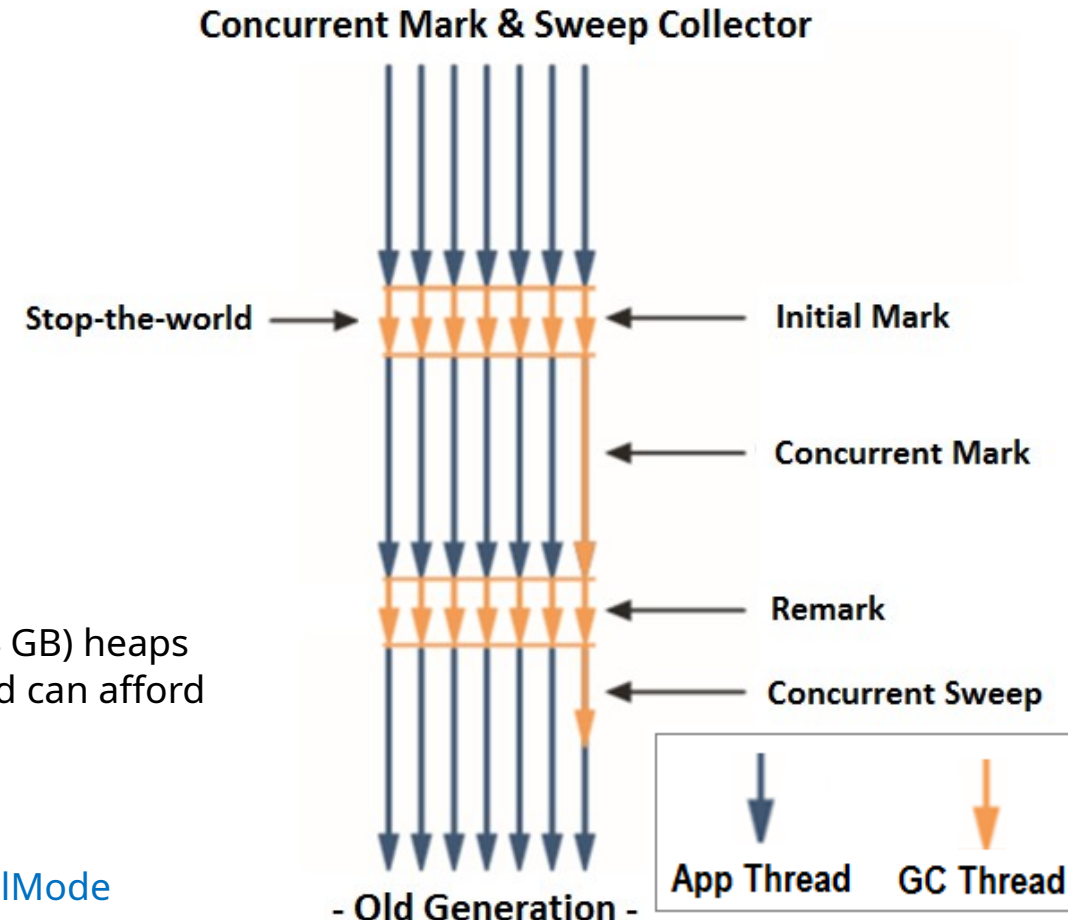
- ◆ does not compact memory area
- ◆ lower object promotion due to free list space (FLS)

When to Use

- ◆ good throughput for 2 or more CPU cores with medium (up to 4 GB) heaps
- ◆ for applications that need shorter garbage collection pauses and can afford to share processor resources with the garbage collector

CMS Collector selection: `-XX:+UseConcMarkSweepGC`

- ◆ concurrent phases are done incrementally `-XX:+CMSIncrementalMode`



Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

5. Garbage First (G1) GC

Young Generation

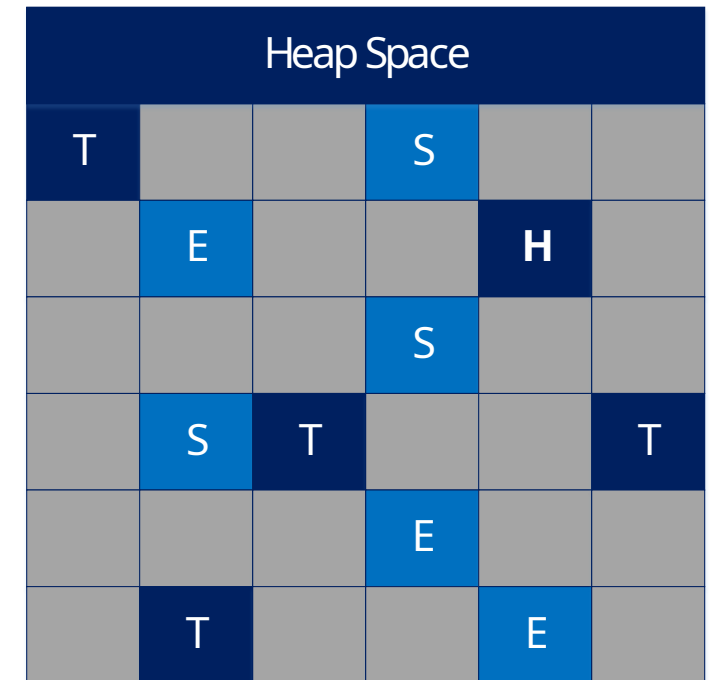
- ◆ Stop-the-world and parallel

Old Generation

- ◆ Concurrent, parallel and copying collector

Split memory into **Regions** (1MB to 32 MB)

- ◆ neither the Young nor the Old generation has to be contiguous
- ◆ regions are dynamically resizable and provide finer granularity



Evolution of Garbage Collection in Java Virtual Machine

Garbage Collectors

5. Garbage First (G1) GC

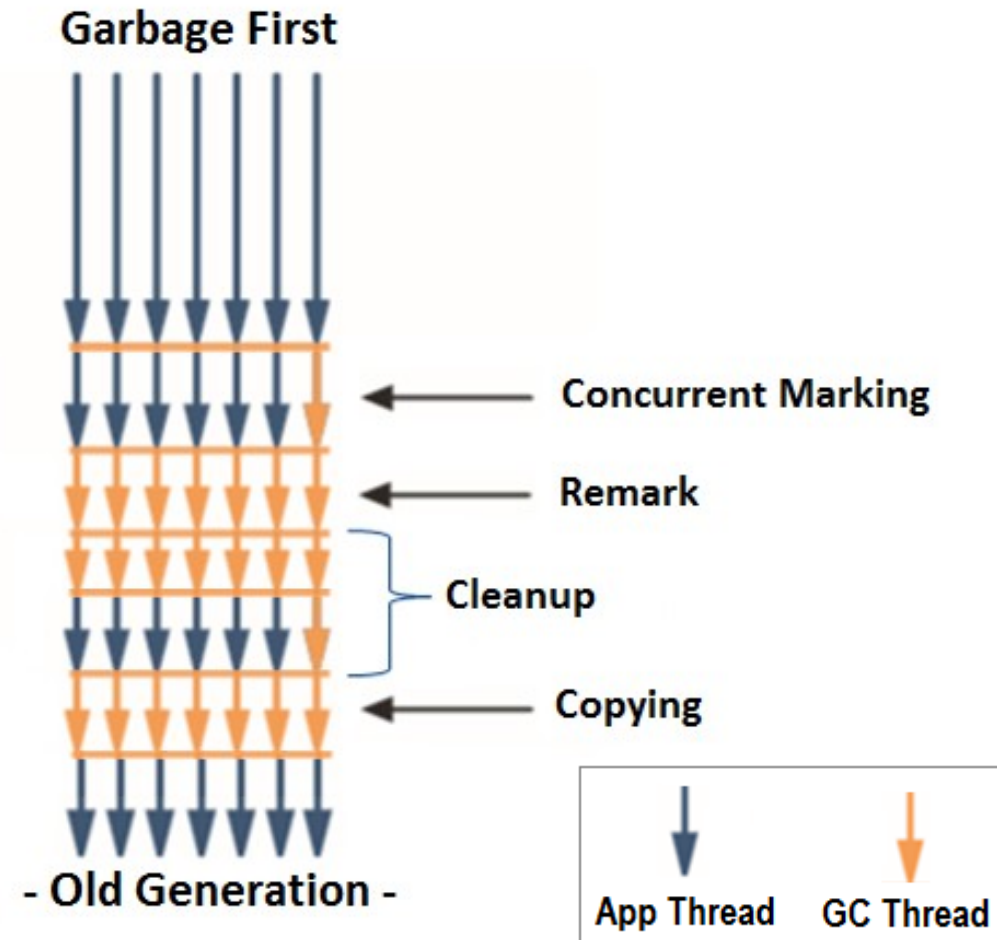
G1 is planned as the long term replacement for CMS

- ♦ is a compacting collector
- ♦ offers more predictable garbage collection pauses than the CMS collector

When to Use

- ♦ good throughput for 2-4 CPU cores with medium-high (4-8 GB) heaps

G1 Collector selection: `-XX:+UseG1GC`



Evolution of Garbage Collection in Java Virtual Machine

Advices of choosing the appropriate Garbage Collector

- ♦ Start with **Parallel Old GC**

Parallel Old GC offers the fastest minor GC times due to **Promotional Local Allocation Buffers**

If you can avoid full GCs, you'll likely achieve

- ♦ the best throughput
- ♦ smallest footprint
- ♦ lowest latency

- ♦ Move to **CMS** or **G1** if needed (especially for Tenured generation collections)

CMS minor GC times are slower due to promotion into **Free Lists Space**

CMS full GC is in essence quicker due to concurrent Tenured collection

G1 full GC offers better results due to “partial” and concurrent Tenured collection and it also avoids fragmentation

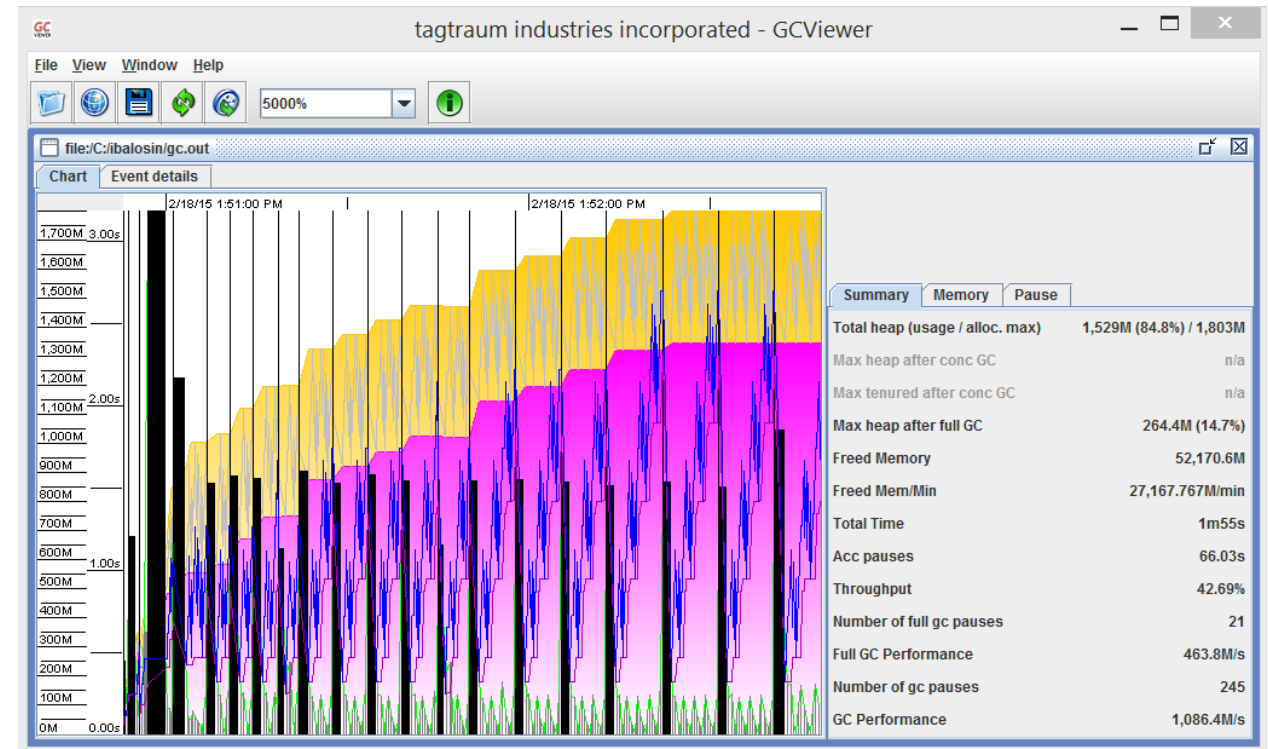
Evolution of Garbage Collection in Java Virtual Machine

➤ Garbage Collection Viewer

- ◆ <https://github.com/chewiebug/GCViewer>

Support for

- ◆ Sun / Oracle
 - ◆ IBM
 - ◆ BEA JRockit
- ◆ Oracle HotSpot:



`-Xloggc:<file> -XX:+PrintGCDetails -XX:+PrintGCDateStamps`

Evolution of Garbage Collection in Java Virtual Machine

Garbage Collection friendly programming

♦ -XX:+DoEscapeAnalysis

"Java object instances are allocated on the Heap" – not really true 😊

Escape Analysis allocates Java objects on the stack only if the an object has a local scope and does not escape

```
static double sum(int[] values) {  
  
    double result = 0;  
    for (int i = 1; i < values.length; i++) {  
        DoubleSlot slot = new DoubleSlot(values[i - 1], values[i]);  
        result += slotValue(slot); // slot.value1 + slot.value2  
    }  
  
    return result;  
}
```

♦ Data Structure Re-sizing

avoid re-sizing of array backed collections / StringBuilder / StringBuffer

- ♦ re-sizing leads to unnecessary object allocation (*fragmentation*)
- ♦ Advice: use the constructor with an explicit size for the backing array

Evolution of Garbage Collection in Java Virtual Machine

Quizz

```
class MyImpl extends ClassWithFinalizer {  
    private byte[] buffer = new byte[1024 * 1024 * 2];  
    ...  
}
```

```
class MyImpl {  
    private ClassWithFinalier classWithFinalizer;  
    private byte[] buffer = new byte[1024 * 1024 * 2];  
    ...  
}
```

1. What is the object retention consequences if **ClassWithFinalizer** has a `finalizer()` method?
2. How to lower the object retention?

Evolution of Garbage Collection in Java Virtual Machine

Garbage Collection friendly programming

- ♦ Avoid to use **finalize()**

is called only once when an object is about to get garbage collected

requires at least 2 Garbage Collector cycles and these cycles are slower

Advice:

- ♦ avoid **finalize()** for the “correct” operation of your application
- ♦ **finalize()** should **only** be used for cleanup of (usually non-Java) resources

- ♦ **Inner Classes**

have an implicit reference to the outer instance

- ♦ can potentially increase object retention
more live objects at GC time , so increased GC duration

Evolution of Garbage Collection in Java Virtual Machine

Bibliography

- *Java Performance* - Charlie Hunt, Binu John
- *Java Performance. The Definitive Guide* - Scott Oaks
- [InfoQ.com](#): *Extreme Performance with Java* - Charlie Hunt

THANK YOU

Appendix

Evolution of Garbage Collection in Java Virtual Machine

Read Garbage Collector Output

- ♦ **Minor Garbage Collection**

```
0.150: [GC
      [PSYoungGen: 142816K->10752K(142848K)] 246648K->243136K(375296K),
      0,0935090 secs
      ]
      [Times: user=0,55 sys=0,10, real=0,09 secs]
```

- ♦ **0.150:** is the start time of the collection since the VM started
- ♦ **[Times: user=0.55 sys=0.10 real=0.09 secs]**
 - User** is the amount of CPU time spent in user-mode code (outside the kernel) *within* the process
 - Sys** is the amount of CPU time spent in the kernel within the process (system calls *within the kernel*)
 - Real** is wall clock time – duration from start to finish of the collection
- ♦ Note: **User** + **Sys** = how much actual CPU time your process used
Note that this is across all CPUs, so if the process *has multiple threads* it could potentially exceed the wall clock time reported by **Real**

Evolution of Garbage Collection in Java Virtual Machine

Read Garbage Collector Output

- ♦ Full Garbage Collection

0.150: [Full GC

[PSYoungGen: 10752K->9707K(142848K)]

[ParOldGen: 232384K->232244K(485888K)] 243136K->241951K(628736K)

[PSPermGen: 3162K->3161K(21504K)],

1,5265450 secs

]

[Times: user=10,96 sys=0,06, real=1,53 secs]

Evolution of Garbage Collection in Java Virtual Machine

Quizz

- ◆ Let's suppose `createMemoryLeak()` throws `OutOfMemoryError` (OOME)

```
try {  
    createMemoryLeak();  
} catch (OutOfMemoryError e) {  
    e.printStackTrace();  
    System.out.println("Explicitly call Garbage Collector ");  
    System.gc();  
    System.out.println("Life after called Garbage Collector ");  
}
```

1. What happens before `OOME` is thrown?
2. Can the program execution continue outside of `try - catch` ?